```
import streamlit as st
import pandas as pd
import numpy as np
import plotly.express as px
import plotly.graph_objects as go

# ── 1. Page Configuration ──────────────────────────────────────
st.set_page_config(
    page_title="NBA Market Efficiency",
    page_icon="🏀",
    layout="wide",
    initial_sidebar_state="expanded"
)

# ── 2. Global Styling ──────────────────────────────────────────
# Navy-layered dark theme with amber + cyan accents
BG_PAGE    = "#0d1117"   # deepest background
BG_CARD    = "#10161f"   # card / chart surface
BG_BORDER  = "#1e2d3d"   # borders & grid lines
C_AMBER    = "#f59e0b"   # primary accent
C_CYAN     = "#00e5ff"   # scatter / data points
C_SKY      = "#38bdf8"   # hover highlights / callouts
C_GREEN    = "#22c55e"   # positive / over
C_RED      = "#ef4444"   # negative / under
C_MUTED    = "#4a6080"   # muted labels
C_TEXT     = "#e2e8f0"   # primary text
C_GOLD     = "#D4AF37"   # breakeven / reference lines (keep legacy feel)

st.markdown(f"""
<style>
/* ── Page background ── */
.stApp {{
    background-color: {BG_PAGE};
}}

/* ── Sidebar ── */
[data-testid="stSidebar"] {{
    background-color: {BG_CARD};
```

```
    border-right: 1px solid {BG_BORDER};
}}
[data-testid="stSidebar"] .block-container {{
    padding-top: 1.5rem;
}}

/* ── All text inside sidebar ── */
[data-testid="stSidebar"] * {{
    color: {C_TEXT} !important;
}}

/* ── Sidebar section headers (markdown bold) ── */
[data-testid="stSidebar"] h3 {{
    color: {C_MUTED} !important;
    font-size: 10px !important;
    letter-spacing: 0.08em;
    text-transform: uppercase;
    margin-bottom: 4px;
}}

/* ── Tabs ── */
.stTabs [data-baseweb="tab-list"] {{
    gap: 0px;
    background-color: {BG_CARD};
    border-radius: 8px 8px 0 0;
    border-bottom: 1px solid {BG_BORDER};
    padding: 0 8px;
}}
.stTabs [data-baseweb="tab"] {{
    color: {C_MUTED} !important;
    background: transparent !important;
    border-bottom: 2px solid transparent !important;
    font-size: 13px;
    padding: 10px 16px;
}}
.stTabs [aria-selected="true"] {{
    color: {C_AMBER} !important;
    border-bottom: 2px solid {C_AMBER} !important;
    background: transparent !important;
}}
```

```
.stTabs [data-baseweb="tab-panel"] {{
    background-color: {BG_PAGE};
    padding-top: 1.25rem;
}}

/* ── Metric cards ── */
[data-testid="stMetric"] {{
    background-color: {BG_CARD};
    border: 1px solid {BG_BORDER};
    border-radius: 10px;
    padding: 14px 18px;
}}
[data-testid="stMetricLabel"] {{
    color: {C_MUTED} !important;
    font-size: 11px !important;
    text-transform: uppercase;
    letter-spacing: 0.06em;
}}
[data-testid="stMetricValue"] {{
    color: {C_TEXT} !important;
    font-size: 26px !important;
    font-weight: 500 !important;
}}
[data-testid="stMetricDelta"] {{
    font-size: 11px !important;
}}

/* ── Section sub-label helper class ── */
.section-label {{
    font-size: 11px;
    color: {C_MUTED};
    text-transform: uppercase;
    letter-spacing: 0.08em;
    margin-bottom: 6px;
    margin-top: 4px;
}}

/* ── Insight callout cards ── */
.insight-card {{
    background: {BG_CARD};
```

```
    border: 1px solid {BG_BORDER};
    border-left: 3px solid {C_AMBER};
    border-radius: 8px;
    padding: 12px 16px;
    font-size: 12px;
    color: {C_MUTED};
    height: 100%;
}}
.insight-card strong {{
    color: {C_AMBER};
    display: block;
    font-size: 13px;
    margin-bottom: 4px;
}}

/* ── KPI accent top-bar ── */
.kpi-amber {{ border-top: 2px solid {C_AMBER} !important; }}
.kpi-teal  {{ border-top: 2px solid #14b8a6 !important; }}
.kpi-sky   {{ border-top: 2px solid {C_SKY} !important; }}
.kpi-green {{ border-top: 2px solid {C_GREEN} !important; }}

/* ── Plotly chart containers ── */
.stPlotlyChart {{
    background: {BG_CARD};
    border: 1px solid {BG_BORDER};
    border-radius: 10px;
    overflow: hidden;
    padding: 4px;
}}

/* ── Dividers ── */
hr {{
    border-color: {BG_BORDER} !important;
    margin: 1.25rem 0 !important;
}}

/* ── Source attribution ── */
.source-tag {{
    font-size: 10px;
    color: {C_MUTED};
```

```
    text-align: right;
    margin-top: 16px;
    letter-spacing: 0.04em;
}}

/* ── Selectbox / Radio / Slider colors ── */
[data-testid="stSelectbox"] > div,
[data-testid="stMultiSelect"] > div {{
    background-color: {BG_CARD} !important;
    border-color: {BG_BORDER} !important;
    color: {C_TEXT} !important;
}}
</style>
""", unsafe_allow_html=True)

# ── Plotly base layout shared by all charts
# ──────────────────────────────────────
PLOTLY_BASE = dict(
    template="plotly_dark",
    paper_bgcolor=BG_CARD,
    plot_bgcolor=BG_PAGE,
    font=dict(color=C_TEXT, size=13),
    margin=dict(t=36, b=36, l=16, r=16),
    xaxis=dict(
        gridcolor=BG_BORDER,
        zerolinecolor=BG_BORDER,
        showgrid=True,
    ),
    yaxis=dict(
        gridcolor=BG_BORDER,
        zerolinecolor=BG_BORDER,
        showgrid=True,
    ),
    hoverlabel=dict(
        bgcolor=BG_CARD,
        bordercolor=C_AMBER,
        font_color=C_TEXT,
        font_size=12,
    ),
)
```

```
# ── 3. Data Engine ──────────────────────────────────────────────────────
@st.cache_data
def load_and_prep_data():
    df = pd.read_csv('oddsData.csv')
    df['date'] = pd.to_datetime(df['date'])

    team_mapping = {
        'Atlanta': 'Hawks', 'Boston': 'Celtics', 'Brooklyn': 'Nets', 'New
Jersey': 'Nets',
        'Charlotte': 'Hornets', 'Chicago': 'Bulls', 'Cleveland':
'Cavaliers', 'Dallas': 'Mavericks',
        'Denver': 'Nuggets', 'Detroit': 'Pistons', 'Golden State':
'Warriors', 'Houston': 'Rockets',
        'Indiana': 'Pacers', 'LA Clippers': 'Clippers', 'LA Lakers':
'Lakers', 'Memphis': 'Grizzlies',
        'Miami': 'Heat', 'Milwaukee': 'Bucks', 'Minnesota':
'Timberwolves', 'New Orleans': 'Pelicans',
        'New York': 'Knicks', 'Oklahoma City': 'Thunder', 'Orlando':
'Magic', 'Philadelphia': '76ers',
        'Phoenix': 'Suns', 'Portland': 'Trail Blazers', 'Sacramento':
'Kings', 'San Antonio': 'Spurs',
        'Toronto': 'Raptors', 'Utah': 'Jazz', 'Washington': 'Wizards',
'Seattle': 'SuperSonics'
    }

    df['team']     = df['team'].map(team_mapping).fillna(df['team'])
    df['opponent'] =
df['opponent'].map(team_mapping).fillna(df['opponent'])

    df['actual_margin'] = df['score'] - df['opponentScore']
    df['adjusted_score'] = df['score'] + df['spread']
    df['su_win'] = np.where(df['score'] > df['opponentScore'], 1, 0)

    conditions_ats = [
        df['adjusted_score'] > df['opponentScore'],
        df['adjusted_score'] < df['opponentScore'],
        df['adjusted_score'] == df['opponentScore']
    ]
```

```
    df['ats_result']      = np.select(conditions_ats, ['Win', 'Loss',
'Push'], default='Unknown')
    df['margin_of_cover'] = df['adjusted_score'] - df['opponentScore']

    df['actual_total'] = df['score'] + df['opponentScore']
    conditions_ou = [
        df['actual_total'] > df['total'],
        df['actual_total'] < df['total'],
        df['actual_total'] == df['total']
    ]
    df['ou_result'] = np.select(conditions_ou, ['Over', 'Under', 'Push'],
default='Unknown')

    def calc_implied_prob(ml):
        if pd.isna(ml): return np.nan
        if ml < 0: return abs(ml) / (abs(ml) + 100)
        else:       return 100 / (ml + 100)

    df['implied_win_prob'] = df['moneyLine'].apply(calc_implied_prob)

    df_games = df[df['home/visitor'] == 'vs'].copy()
    return df, df_games

with st.spinner("Loading 16 Seasons of NBA Data..."):
    df_teams, df_games = load_and_prep_data()

# ── 4. Header
─────────────────────────────────────────────────────────────────
st.title("🏀 NBA Betting Market Efficiency")
st.markdown(
    "<p style='color:#4a6080;font-size:13px;margin-top:-12px;'>"
    "16 seasons of historical betting odds — identify market gaps and
inefficiencies  ·  "
    "<span style='color:#38bdf8;'>Christopher Treasure ·
Kaggle</span></p>",
    unsafe_allow_html=True
)

# ── 5. Sidebar
─────────────────────────────────────────────────────────────────
```

```python
st.sidebar.markdown("### 🕹️ Control Center")

min_year = int(df_teams['season'].min())
max_year = int(df_teams['season'].max())
selected_seasons = st.sidebar.slider(
    "Season Range", min_value=min_year, max_value=max_year,
value=(min_year, max_year)
)

all_teams = sorted(df_teams['team'].unique())
selected_teams = st.sidebar.multiselect("Filter by Team(s)",
options=all_teams, default=[])

venue_filter = st.sidebar.radio("Venue", ["All", "Home", "Away"])

st.sidebar.markdown("---")
st.sidebar.markdown(
    f"<div style='font-size:11px;color:{C_MUTED};'>"
    "📊 <b style='color:{C_SKY}'>Data</b>: Christopher Treasure<br>"
    "Kaggle · NBA Odds Dataset<br>"
    f"Seasons: {selected_seasons[0]}-{selected_seasons[1]}</div>",
    unsafe_allow_html=True
)

# ── 6. Filter Logic
─────────────────────────────────────────────────────────────

mask_games = (
    (df_games['season'] >= selected_seasons[0]) &
    (df_games['season'] <= selected_seasons[1])
)
if selected_teams:
    if venue_filter == "Home":
        mask_games &= df_games['team'].isin(selected_teams)
    elif venue_filter == "Away":
        mask_games &= df_games['opponent'].isin(selected_teams)
    else:
        mask_games &= (df_games['team'].isin(selected_teams) |
df_games['opponent'].isin(selected_teams))
filtered_games = df_games[mask_games]
```

```
mask_teams = (
    (df_teams['season'] >= selected_seasons[0]) &
    (df_teams['season'] <= selected_seasons[1])
)
if selected_teams:
    mask_teams &= df_teams['team'].isin(selected_teams)
if venue_filter == "Home":
    mask_teams &= (df_teams['home/visitor'] == 'vs')
elif venue_filter == "Away":
    mask_teams &= (df_teams['home/visitor'] == '@')
filtered_teams = df_teams[mask_teams]

# ── 7. Tabs ──────────────────────────────────────────────────────────────
tab1, tab2, tab3, tab4 = st.tabs([
    "📈  Market Macro",
    "📊  Team Edge",
    "💰  Strategy Simulator",
    "📖  Playbook",
])

# ═══════════════════════════════════════════════════════════════════════
# TAB 1 — MARKET MACRO
# ═══════════════════════════════════════════════════════════════════════
with tab1:
    if not filtered_games.empty:
        # ── KPI helpers ──────────────────────────────────────────────
        avg_total   = filtered_games['actual_total'].mean()
        over_rate   = (filtered_games['ou_result'] == 'Over').mean() * 100
        home_margin = filtered_games['actual_margin'].mean()
        n_games     = len(filtered_games)

        # Trend: compare first vs last 3 seasons in selection
        seasons_sorted = sorted(filtered_games['season'].unique())
        if len(seasons_sorted) >= 6:
```

```
            early =
filtered_games[filtered_games['season'].isin(seasons_sorted[:3])]
            late  =
filtered_games[filtered_games['season'].isin(seasons_sorted[-3:])]
            total_trend  = late['actual_total'].mean() -
early['actual_total'].mean()
            margin_trend = late['actual_margin'].mean() -
early['actual_margin'].mean()
        else:
            total_trend  = None
            margin_trend = None

        push_rate  = (filtered_games['ou_result'] == 'Push').mean() * 100
        under_rate = (filtered_games['ou_result'] == 'Under').mean() * 100
        home_wins  = (filtered_games['su_win'] == 1).mean() * 100

        # ── KPI Row
──────────────────────────────────────────────────────
        c1, c2, c3, c4 = st.columns(4)
        with c1:
            st.markdown('<div class="kpi-amber">', unsafe_allow_html=True)
            total_delta_str = f"+{total_trend:.1f} pts vs. early seasons"
if total_trend and total_trend > 0 else (f"{total_trend:.1f} pts vs. early
seasons" if total_trend else None)
            st.metric("Avg. Game Total", f"{avg_total:.1f} pts",
delta=total_delta_str)
            st.markdown(
                f"<p
style='font-size:11px;color:{C_MUTED};margin-top:-8px;'>"
                f"Over: {over_rate:.1f}% · Under: {under_rate:.1f}% ·
Push: {push_rate:.1f}%</p>",
                unsafe_allow_html=True
            )
            st.markdown('</div>', unsafe_allow_html=True)
        with c2:
            st.markdown('<div class="kpi-teal">', unsafe_allow_html=True)
            over_delta = over_rate - 50
            st.metric("Over Rate", f"{over_rate:.1f}%",
delta=f"{over_delta:+.1f}% vs. fair 50/50")
            st.markdown(
```

```
                f"<p
style='font-size:11px;color:{C_MUTED};margin-top:-8px;'>"
                f"{'Slight Under edge' if over_rate < 50 else 'Slight Over
edge'} across dataset</p>",
                unsafe_allow_html=True
            )
            st.markdown('</div>', unsafe_allow_html=True)
        with c3:
            st.markdown('<div class="kpi-sky">', unsafe_allow_html=True)
            margin_delta_str = (
                f"{margin_trend:+.1f} pts trend" if margin_trend is not
None else None
            )
            st.metric("Home Win Margin", f"{home_margin:.1f} pts",
delta=margin_delta_str)
            st.markdown(
                f"<p
style='font-size:11px;color:{C_MUTED};margin-top:-8px;'>"
                f"Home SU win rate: {home_wins:.1f}%</p>",
                unsafe_allow_html=True
            )
            st.markdown('</div>', unsafe_allow_html=True)
        with c4:
            st.markdown('<div class="kpi-green">', unsafe_allow_html=True)
            st.metric("Games Analyzed", f"{n_games:,}")
            st.markdown(
                f"<p
style='font-size:11px;color:{C_MUTED};margin-top:-8px;'>"
                f"Seasons {selected_seasons[0]}-{selected_seasons[1]} · "
                f"{len(filtered_games['team'].unique())} teams</p>",
                unsafe_allow_html=True
            )
            st.markdown('</div>', unsafe_allow_html=True)

        st.divider()

        # ── Scatter: Expected vs. Actual
──────────────────────────────────

        st.markdown('<p class="section-label">Expected Totals vs. Actual
Scores</p>', unsafe_allow_html=True)
```

```
        fig_scatter = px.scatter(
            filtered_games,
            x="total", y="actual_total",
            opacity=0.45,
            color_discrete_sequence=[C_CYAN],
            hover_name="date",
            hover_data={"total": True, "actual_total": True, "team": True,
"opponent": True},
        )
        fig_scatter.add_shape(
            type="line", line=dict(dash='dash', color=C_RED, width=2),
            x0=filtered_games['total'].min(),
y0=filtered_games['total'].min(),
            x1=filtered_games['total'].max(),
y1=filtered_games['total'].max()
        )
        fig_scatter.update_traces(
            marker=dict(size=5, line=dict(width=0)),
            selected=dict(marker=dict(color=C_AMBER, size=10, opacity=1)),
            unselected=dict(marker=dict(opacity=0.15)),
        )
        fig_scatter.update_layout(
            **PLOTLY_BASE,
            clickmode="event+select",
            xaxis_title="Vegas Predicted Total",
            yaxis_title="Actual Combined Score",
            annotations=[dict(
                x=filtered_games['total'].max() * 0.98,
                y=filtered_games['total'].min() * 1.02,
                text="Perfect calibration line",
                showarrow=False, font=dict(color=C_RED, size=11)
            )]
        )
        st.plotly_chart(fig_scatter, use_container_width=True)

        c_left, c_right = st.columns(2)

        with c_left:
            st.markdown('<p class="section-label">Market Outcome % by
Season</p>', unsafe_allow_html=True)
```

```
            ou_trend  = filtered_games.groupby(['season',
'ou_result']).size().reset_index(name='count')
            ou_pivot  = ou_trend.pivot(index='season',
columns='ou_result', values='count').fillna(0)
            ou_pct    = ou_pivot.div(ou_pivot.sum(axis=1), axis=0) * 100
            ou_pct    = ou_pct.reset_index().melt(id_vars='season',
value_name='Percentage')
            fig_area  = px.area(
                ou_pct, x="season", y="Percentage", color="ou_result",
                color_discrete_map={"Over": C_GREEN, "Under": C_RED,
"Push": C_MUTED},
            )
            fig_area.add_hline(y=50, line_dash="dot", line_color="white",
opacity=0.4,
                               annotation_text="50%",
annotation_position="right")
            fig_area.update_layout(**PLOTLY_BASE, xaxis_title="Season",
yaxis_title="% of Games",
                                   legend=dict(orientation="h", y=1.08))
            st.plotly_chart(fig_area, use_container_width=True)

        with c_right:
            st.markdown('<p class="section-label">Home-Court Advantage
Decay</p>', unsafe_allow_html=True)
            home_adv =
filtered_games.groupby('season')['actual_margin'].mean().reset_index()
            fig_line = px.line(
                home_adv, x="season", y="actual_margin", markers=True,
                color_discrete_sequence=[C_AMBER],
            )
            fig_line.add_hline(y=0, line_dash="dot", line_color="white",
opacity=0.4,
                               annotation_text="No advantage",
annotation_position="right")
            fig_line.update_traces(
                line=dict(width=2.5),
                marker=dict(size=7, color=C_AMBER, line=dict(width=1.5,
color=BG_PAGE))
            )
```

```
            fig_line.update_layout(**PLOTLY_BASE, xaxis_title="Season",
yaxis_title="Avg. Home Win Margin (pts)")
            st.plotly_chart(fig_line, use_container_width=True)

        # ── Insight strip
──────────────────────────────────────────────────
        st.divider()
        i1, i2, i3 = st.columns(3)
        with i1:
            edge = "Under" if over_rate < 50 else "Over"
            st.markdown(
                f'<div class="insight-card"><strong>📉 {edge} edge
detected</strong>'
                f'Over rate sits at {over_rate:.1f}% —
{abs(over_rate-50):.1f}% away from break-even. '
                f'The market has slightly mis-set totals across
{n_games:,} games.</div>',
                unsafe_allow_html=True
            )
        with i2:
            st.markdown(
                f'<div class="insight-card"><strong>🏠 Home premium:
{home_margin:.1f} pts</strong>'
                f'{("Declining trend detected — road teams may be
underpriced by Vegas." if margin_trend and margin_trend < -0.3 else
"Home-court edge remains relatively stable across the selected window.")}'
                f'</div>',
                unsafe_allow_html=True
            )
        with i3:
            st.markdown(
                f'<div class="insight-card"><strong>📈 Scoring pace
rising</strong>'
                f'Average game total across the dataset is {avg_total:.1f}
pts. '
                f'{"Vegas tends to lag behind scoring surges, creating
Over windows early in a trend shift." if total_trend and total_trend > 2
else "Totals have remained broadly stable in the selected
window."}</div>',
                unsafe_allow_html=True
```

```
            )

        st.markdown('<p class="source-tag">Source: Christopher Treasure · NBA Odds Dataset · Kaggle</p>', unsafe_allow_html=True)
    else:
        st.warning("No data available for the selected filters.")


#
═══════════════════════════════════════════════════════════════════════
══════
# TAB 2 — TEAM EDGE
#
═══════════════════════════════════════════════════════════════════════
══════
with tab2:
    if not filtered_teams.empty:
        ats_valid  =
filtered_teams[filtered_teams['ats_result'].isin(['Win', 'Loss'])]
        team_stats =
ats_valid.groupby('team')['ats_result'].value_counts().unstack(fill_value=
0).reset_index()

        if 'Win'  not in team_stats.columns: team_stats['Win']  = 0
        if 'Loss' not in team_stats.columns: team_stats['Loss'] = 0

        team_stats['Games']       = team_stats['Win'] + team_stats['Loss']
        team_stats['ATS_Win_Pct'] = (team_stats['Win'] /
team_stats['Games']) * 100
        team_stats['Avg_Margin']  =
filtered_teams.groupby('team')['margin_of_cover'].mean().values
        team_stats                = team_stats[team_stats['Games'] >= 10]

        display_teams = (
            team_stats.sort_values('ATS_Win_Pct',
ascending=False).head(15)
            if not selected_teams and len(team_stats) > 15
            else team_stats.sort_values('ATS_Win_Pct', ascending=False)
        )
```

```
        display_teams['Color_ATS']    =
np.where(display_teams['ATS_Win_Pct'] >= 52.4, C_GREEN, C_RED)
        display_teams['Color_Margin'] =
np.where(display_teams['Avg_Margin']  >= 0,    C_GREEN, C_RED)

        # ── KPI Row
────────────────────────────────────────────────────────
        best_ats  =
display_teams.loc[display_teams['ATS_Win_Pct'].idxmax()]
        worst_ats =
display_teams.loc[display_teams['ATS_Win_Pct'].idxmin()]
        best_cov  =
display_teams.loc[display_teams['Avg_Margin'].idxmax()]
        above_be  = (display_teams['ATS_Win_Pct'] >= 52.4).sum()

        k1, k2, k3, k4 = st.columns(4)
        with k1:
            st.markdown('<div class="kpi-green">', unsafe_allow_html=True)
            st.metric("Best ATS Team", best_ats['team'],
delta=f"{best_ats['ATS_Win_Pct']:.1f}% cover rate")
            st.markdown(
                f"<p
style='font-size:11px;color:{C_MUTED};margin-top:-8px;'>"
                f"{int(best_ats['Win'])}W - {int(best_ats['Loss'])}L ·
{int(best_ats['Games'])} games</p>",
                unsafe_allow_html=True
            )
            st.markdown('</div>', unsafe_allow_html=True)
        with k2:
            st.markdown('<div class="kpi-amber">', unsafe_allow_html=True)
            st.metric("Best Cover Margin", best_cov['team'],
delta=f"+{best_cov['Avg_Margin']:.2f} pts avg")
            st.markdown(
                f"<p
style='font-size:11px;color:{C_MUTED};margin-top:-8px;'>"
                f"Consistently beats spread by widest margin</p>",
                unsafe_allow_html=True
            )
            st.markdown('</div>', unsafe_allow_html=True)
        with k3:
```

```
            st.markdown('<div class="kpi-sky">', unsafe_allow_html=True)
            st.metric("Teams Above Breakeven", f"{above_be} /
{len(display_teams)}")
            st.markdown(
                f"<p
style='font-size:11px;color:{C_MUTED};margin-top:-8px;'>"
                f"Breakeven ATS rate = 52.4% at -110 juice</p>",
                unsafe_allow_html=True
            )
            st.markdown('</div>', unsafe_allow_html=True)
        with k4:
            st.markdown('<div class="kpi-amber"
style="border-top-color:#ef4444!important">', unsafe_allow_html=True)
            st.metric("Worst ATS Team", worst_ats['team'],
delta=f"{worst_ats['ATS_Win_Pct']:.1f}% cover rate",
delta_color="inverse")
            st.markdown(
                f"<p
style='font-size:11px;color:{C_MUTED};margin-top:-8px;'>"
                f"{int(worst_ats['Win'])}W - {int(worst_ats['Loss'])}L ·
{int(worst_ats['Games'])} games</p>",
                unsafe_allow_html=True
            )
            st.markdown('</div>', unsafe_allow_html=True)

        st.divider()

        col_t1, col_t2 = st.columns(2)
        with col_t1:
            st.markdown('<p class="section-label">ATS Profitability
Leaderboard</p>', unsafe_allow_html=True)
            sorted_ats = display_teams.sort_values('ATS_Win_Pct',
ascending=True)
            fig_bar1 = px.bar(
                sorted_ats, x="ATS_Win_Pct", y="team", orientation='h',
                text=sorted_ats['ATS_Win_Pct'].apply(lambda x:
f"{x:.1f}%"),
            )
            fig_bar1.update_traces(
                marker_color=sorted_ats['Color_ATS'].tolist(),
```

```
                textposition='inside', textfont=dict(color='white',
size=13)
            )
            fig_bar1.add_vline(
                x=52.4, line_dash="dash", line_color=C_GOLD, line_width=2,
                annotation_text="Breakeven 52.4%",
                annotation_position="bottom right",
                annotation_font=dict(color=C_GOLD, size=11)
            )
            fig_bar1.update_layout(**PLOTLY_BASE)
            fig_bar1.update_layout(margin=dict(t=16, b=16, l=16, r=16))
            fig_bar1.update_xaxes(showgrid=False, range=[30,
display_teams['ATS_Win_Pct'].max() + 6], title="")
            fig_bar1.update_yaxes(showgrid=False, title="")
            st.plotly_chart(fig_bar1, use_container_width=True)

        with col_t2:
            st.markdown('<p class="section-label">Average Margin of
Cover</p>', unsafe_allow_html=True)
            sorted_margin = display_teams.sort_values('Avg_Margin',
ascending=True)
            fig_bar2 = px.bar(
                sorted_margin, x="Avg_Margin", y="team", orientation='h',
                text=sorted_margin['Avg_Margin'].apply(lambda x:
f"{x:.2f}"),
            )
            fig_bar2.update_traces(
                marker_color=sorted_margin['Color_Margin'].tolist(),
                textposition='outside', textfont=dict(size=13)
            )
            fig_bar2.add_vline(x=0, line_dash="solid", line_color="white",
line_width=1, opacity=0.4)
            fig_bar2.update_layout(**PLOTLY_BASE)
            fig_bar2.update_layout(margin=dict(t=16, b=16, l=16, r=16))
            fig_bar2.update_xaxes(showgrid=False, title="")
            fig_bar2.update_yaxes(showgrid=False, title="")
            st.plotly_chart(fig_bar2, use_container_width=True)

        st.divider()
```

```
        st.markdown('<p class="section-label">Season-by-Season Consistency
Heatmap</p>', unsafe_allow_html=True)
        heatmap_data = (
            ats_valid.groupby(['team', 'season'])['ats_result']
            .value_counts().unstack(fill_value=0).reset_index()
        )
        heatmap_data['Win_Pct'] = (
            heatmap_data['Win'] / (heatmap_data['Win'] +
heatmap_data.get('Loss', 0))
        ) * 100
        heatmap_pivot = heatmap_data.pivot(index='team', columns='season',
values='Win_Pct').loc[display_teams['team']]

        fig_heat = px.imshow(
            heatmap_pivot,
            labels=dict(x="Season", y="Team", color="ATS Win %"),
            color_continuous_scale=[(0, C_RED), (0.5, BG_CARD), (1,
C_GREEN)],
            zmin=35, zmax=70, aspect="auto",
        )
        fig_heat.update_layout(
            **PLOTLY_BASE,
            coloraxis_colorbar=dict(
                title="ATS Win %",
                tickvals=[35, 52.4, 70],
                ticktext=["35%", "52.4% BE", "70%"],
                len=0.6
            )
        )
        st.plotly_chart(fig_heat, use_container_width=True)

        st.markdown('<p class="source-tag">Source: Christopher Treasure ·
NBA Odds Dataset · Kaggle</p>', unsafe_allow_html=True)
    else:
        st.warning("No data available.")


#
================================================================================
======
```

```
# TAB 3 — STRATEGY SIMULATOR
#
═══════════════════════════════════════════════════════════════════════
══════
with tab3:
    st.markdown(
        f"<p style='color:{C_MUTED};font-size:13px;'>Flat-betting
simulation based on active filters. "
        f"Assumes standard <b style='color:{C_TEXT}'>-110 sportsbook
odds</b> "
        f"(risk 1.1U to win 1.0U).</p>",
        unsafe_allow_html=True
    )

    if not filtered_teams.empty:
        sim_data = filtered_teams.sort_values('date').copy()

        def calculate_units(result):
            if result == 'Win':   return  1.0
            elif result == 'Loss': return -1.1
            else:                  return  0.0

        sim_data['Unit_Profit']      =
sim_data['ats_result'].apply(calculate_units)
        sim_data['Cumulative_Units'] = sim_data['Unit_Profit'].cumsum()

        total_bets   = len(sim_data[sim_data['ats_result'].isin(['Win',
'Loss'])])
        total_profit = sim_data['Cumulative_Units'].iloc[-1] if total_bets
> 0 else 0
        win_rate     = (sim_data['ats_result'] == 'Win').sum() /
total_bets * 100 if total_bets > 0 else 0
        roi          = (total_profit / (total_bets * 1.1)) * 100 if
total_bets > 0 else 0
        max_dd       = (sim_data['Cumulative_Units'] -
sim_data['Cumulative_Units'].cummax()).min()
        best_stretch = sim_data['Unit_Profit'].rolling(20).sum().max()

        if not selected_teams:
```

```
            # If no teams are selected, show the baseline warning about
the red slope
            st.warning(
                "⚠️ **Baseline Warning:** The current view represents a
blind betting strategy across all games. "
                f"The downward slope of **{total_profit:+.1f} Units**
illustrates the mathematical drain of the -110 sportsbook juice over
{total_bets:,} bets. "
                "**Action:** Use the sidebar to filter for specific
high-performing teams (e.g., Thunder, Celtics) to view a profitable
isolated strategy."
            )
        else:
            # If a team IS selected, dynamically evaluate if it's a
winning or losing strategy
            teams_str = ", ".join(selected_teams)
            if total_profit >= 0:
                st.success(f"🎯 **Targeted Strategy:** Betting blindly on
the **{teams_str}** yields a profitable return of **{total_profit:+.1f}
Units** over {total_bets:,} games, beating the sportsbook juice.")
            else:
                st.error(f"📉 **Targeted Strategy:** Betting blindly on
the **{teams_str}** yields a negative return of **{total_profit:+.1f}
Units** over {total_bets:,} games. Avoid this system.")
        st.write("")

        # ── KPI Row
──────────────────────────────────────────────────────

        s1, s2, s3, s4 = st.columns(4)
        with s1:
            st.markdown('<div class="kpi-sky">', unsafe_allow_html=True)
            st.metric("Total Bets Placed", f"{total_bets:,}")
            st.markdown(
                f"<p
style='font-size:11px;color:{C_MUTED};margin-top:-8px;'>"
                f"Pushes excluded from P&L</p>",
                unsafe_allow_html=True
            )
            st.markdown('</div>', unsafe_allow_html=True)
        with s2:
```

```
            color = "kpi-green" if total_profit >= 0 else "kpi-amber"
            st.markdown(f'<div class="{color}">', unsafe_allow_html=True)
            st.metric(
                "Net Profit (Units)", f"{total_profit:+.1f} U",
                delta="Profitable ✓" if total_profit >= 0 else
"Unprofitable",
                delta_color="normal" if total_profit >= 0 else "inverse"
            )
            st.markdown(
                f"<p
style='font-size:11px;color:{C_MUTED};margin-top:-8px;'>"
                f"Max drawdown: {max_dd:.1f} U</p>",
                unsafe_allow_html=True
            )
            st.markdown('</div>', unsafe_allow_html=True)
        with s3:
            st.markdown('<div class="kpi-teal">', unsafe_allow_html=True)
            st.metric("System Win Rate", f"{win_rate:.1f}%",
delta=f"{win_rate - 52.4:+.1f}% vs. breakeven")
            st.markdown(
                f"<p
style='font-size:11px;color:{C_MUTED};margin-top:-8px;'>"
                f"Breakeven at -110 = 52.4%</p>",
                unsafe_allow_html=True
            )
            st.markdown('</div>', unsafe_allow_html=True)
        with s4:
            st.markdown('<div class="kpi-amber">', unsafe_allow_html=True)
            st.metric("Estimated ROI", f"{roi:+.1f}%")
            st.markdown(
                f"<p
style='font-size:11px;color:{C_MUTED};margin-top:-8px;'>"
                f"Best 20-game stretch: {best_stretch:+.1f} U</p>",
                unsafe_allow_html=True
            )
            st.markdown('</div>', unsafe_allow_html=True)

        st.divider()
```

```
        # ── Equity Curve
────────────────────────────────────────────────────
        st.markdown('<p class="section-label">Cumulative Bankroll Equity
Curve</p>', unsafe_allow_html=True)
        eq_color   = C_GREEN if total_profit >= 0 else C_RED
        fill_color = "rgba(34,197,94,0.08)" if total_profit >= 0 else
"rgba(239,68,68,0.08)"
        fig_equity = px.line(
            sim_data, x="date", y="Cumulative_Units",
            color_discrete_sequence=[eq_color],
            hover_data={"date": True, "team": True, "opponent": True,
"ats_result": True}
        )
        fig_equity.update_traces(
            fill='tozeroy', fillcolor=fill_color,
            line=dict(width=2)
        )
        fig_equity.add_hline(y=0, line_dash="solid", line_color="white",
opacity=0.3)
        fig_equity.update_layout(
            **PLOTLY_BASE,
            xaxis_title="Date", yaxis_title="Cumulative Units (+/-)",
        )
        st.plotly_chart(fig_equity, use_container_width=True)

        # ── Calibration
──────────────────────────────────────────────────
        st.markdown('<p class="section-label">Moneyline Calibration —
Implied Probability vs. Actual Reality</p>', unsafe_allow_html=True)
        st.markdown(
            f"<p
style='font-size:12px;color:{C_MUTED};margin-top:-8px;margin-bottom:8px;'>
"
            f"Grouped by 5% implied probability bins. "
            f"Bubbles <b style='color:{C_TEXT}'>below</b> the amber line =
market overvalued those teams.</p>",
            unsafe_allow_html=True
        )
        bins        = np.arange(0, 1.05, 0.05)
```

```
        sim_data['prob_bin'] = pd.cut(sim_data['implied_win_prob'],
bins=bins)
        calib_data  = sim_data.groupby('prob_bin', observed=True).agg(
            Games=('team', 'count'),
            Expected_Win_Pct=('implied_win_prob', 'mean'),
            Actual_Win_Pct=('su_win', 'mean')
        ).reset_index().dropna()
        calib_data['Expected_Win_Pct'] *= 100
        calib_data['Actual_Win_Pct']   *= 100
        calib_data = calib_data[calib_data['Games'] >= 15]

        fig_calib = px.scatter(
            calib_data, x="Expected_Win_Pct", y="Actual_Win_Pct",
            size="Games", hover_name="Games",
            color_discrete_sequence=[C_CYAN]
        )
        axis_min = max(0,   min(calib_data['Expected_Win_Pct'].min(),
calib_data['Actual_Win_Pct'].min()) - 5)
        axis_max = min(100, max(calib_data['Expected_Win_Pct'].max(),
calib_data['Actual_Win_Pct'].max()) + 5)
        fig_calib.add_shape(
            type="line", line=dict(dash='dash', color=C_GOLD, width=2),
            x0=0, y0=0, x1=100, y1=100
        )
        fig_calib.update_traces(
            marker=dict(opacity=0.75, line=dict(width=1, color=BG_CARD)),
            selected=dict(marker=dict(color=C_AMBER, opacity=1)),
        )
        fig_calib.update_layout(**PLOTLY_BASE, clickmode="event+select")
        fig_calib.update_xaxes(range=[axis_min, axis_max], title="Vegas
Implied Win %")
        fig_calib.update_yaxes(range=[axis_min, axis_max], title="Actual
Straight-Up Win %")
        st.plotly_chart(fig_calib, use_container_width=True)

        st.markdown('<p class="source-tag">Source: Christopher Treasure ·
NBA Odds Dataset · Kaggle</p>', unsafe_allow_html=True)
    else:
        st.warning("No data available for the selected filters.")
```

```
#
═══════════════════════════════════════════════════════════════════════
══════
# TAB 4 — PLAYBOOK
#
═══════════════════════════════════════════════════════════════════════
══════
with tab4:
    st.header("The Playbook: How to Read the Market")
    st.markdown(
        f"<p style='color:{C_MUTED};font-size:13px;'>Select a
visualization to learn its anatomy "
        f"and how to find your betting edge.</p>",
        unsafe_allow_html=True
    )

    with st.expander("📚 Betting 101: Executive Cheat Sheet
(Terminology)"):
        c1, c2, c3, c4 = st.columns(4)
        c1.info("**Against the Spread (ATS)**\n\nBetting on the *margin*
of victory. If a team is -5.5, they must win by 6 or more to 'cover' the
spread.")
        c2.info("**Moneyline (ML)**\n\nBetting on who will win
straight-up, regardless of score. Negative odds (-150) indicate the
favorite.")
        c3.info("**Totals (Over/Under)**\n\nBetting on whether the
combined score of both teams will be higher or lower than Vegas's
predicted number.")
        c4.warning("**The Juice (Vig)**\n\nThe sportsbook's commission.
Standard odds are -110, meaning you must risk $110 to win $100. This is
why the breakeven is 52.4%.")
    st.write("")

    lesson_choice = st.selectbox("Choose a Visual Anatomy Lesson:", [
        "Expected Totals vs. Actual Scores",
        "Market Outcome Percentage",
        "Home-Court Decay",
        "ATS Profitability Leaderboard",
        "Average Margin of Cover",
```

```
        "Season-by-Season Consistency Heatmap",
        "Cumulative Bankroll Equity Curve",
        "Moneyline Calibration"
    ])

    st.divider()
    col_chart, col_text = st.columns([2, 1])

    LESSONS = {
        "Expected Totals vs. Actual Scores": {
            "title": "Anatomy of Calibration",
            "what":  "The red dashed line represents market perfection. A
dot on the line means Vegas predicted the combined score exactly. Click
any dot to highlight it.",
            "how":   "The X-axis is Vegas's pre-game total; the Y-axis is
the real combined score. Hover to see the exact game.",
            "edge":  "Clusters far above the line signal the market is
lagging behind scoring surges — your cue to bet the Over.",
        },
        "Market Outcome Percentage": {
            "title": "Anatomy of Market Shifts",
            "what":  "Stacked bands show the % of games hitting Over
(green), Under (red), or Push per season.",
            "how":   "Aggregates thousands of games per season. The Y-axis
tracks outcome distribution for that year.",
            "edge":  "A growing green band over 3+ seasons means the
league scores faster than Vegas raises totals.",
        },
        "Home-Court Decay": {
            "title": "Anatomy of the Arena",
            "what":  "The amber line shows the average points the home
team wins by each season across the whole league.",
            "how":   "Chronological average home-team point differential.
The dotted white line = break-even (0 pts).",
            "edge":  "A declining line means Vegas still prices home teams
as -3 favorites while the real edge has shrunk — bet the road team.",
        },
        "ATS Profitability Leaderboard": {
            "title": "Anatomy of an Edge",
```

```
            "what":  "The gold line marks 52.4% — the breakeven at
standard -110 juice. Green bars are profitable; red are not.",
            "how":   "Each bar is a team's ATS win rate across your filter
window. Updates live with sidebar changes.",
            "edge":  "Green bars to the right of the gold line are
franchises the public consistently undervalues.",
        },
        "Average Margin of Cover": {
            "title": "Anatomy of Safety",
            "what":  "Green = beat the spread by that margin; Red = failed
to cover by that margin.",
            "how":   "Mean difference between a team's adjusted final
score and the opponent across the selected window.",
            "edge":  "High win rate + wide positive margin = 'Safe Bet' —
they don't just cover, they blow Vegas out.",
        },
        "Season-by-Season Consistency Heatmap": {
            "title": "Anatomy of Trends",
            "what":  "Green cells = profitable ATS season (>52.4%), red =
losing season. Hover for exact percentages.",
            "how":   "Each cell is a single team-season. Color intensity
shows how far from breakeven they were.",
            "edge":  "Avoid alternating red/green teams. Solid green
blocks = sustained market bias worth exploiting.",
        },
        "Cumulative Bankroll Equity Curve": {
            "title": "Anatomy of Profit",
            "what":  "Tracks cumulative profit assuming $110 risked to win
$100 (-110 juice) for every game in your filter.",
            "how":   "+1.0 unit per ATS win; -1.1 units per ATS loss.
Pushes are neutral.",
            "edge":  "A rising green curve proves your filter strategy
holds a mathematical edge over the vig.",
        },
        "Moneyline Calibration": {
            "title": "Anatomy of Value",
            "what":  "The amber diagonal = Vegas expectations. A bubble ON
the line means Vegas was perfectly right.",
            "how":   "Teams grouped by implied win probability (x) vs. how
often they actually won straight-up (y).",
```

```
            "edge":  "Bubbles far BELOW the line = 'Overvalued Darlings' — bet heavily by the public but winning less than implied.",
        },
    }

    info = LESSONS[lesson_choice]

    with col_text:
        st.markdown(
            f"<h3 style='color:{C_TEXT};font-size:16px;font-weight:500;margin-bottom:12px;'>{info['title']}</h3>",
            unsafe_allow_html=True
        )
        st.info(f"🔵 **What you're seeing:** {info['what']}")
        st.warning(f"⚙️ **How it works:** {info['how']}")
        st.success(f"📈 **The edge:** {info['edge']}")

    with col_chart:
        if not filtered_teams.empty and not filtered_games.empty:
            chart_map = {
                "Expected Totals vs. Actual Scores":      ("fig_scatter", fig_scatter),
                "Market Outcome Percentage":              ("fig_area", fig_area),
                "Home-Court Decay":                       ("fig_line", fig_line),
                "ATS Profitability Leaderboard":          ("fig_bar1", fig_bar1),
                "Average Margin of Cover":                ("fig_bar2", fig_bar2),
                "Season-by-Season Consistency Heatmap":   ("fig_heat", fig_heat),
                "Cumulative Bankroll Equity Curve":       ("fig_equity", fig_equity),
                "Moneyline Calibration":                  ("fig_calib", fig_calib),
            }
            _, fig = chart_map[lesson_choice]
```

```
            st.plotly_chart(fig, use_container_width=True,
key=f"guide_{lesson_choice[:8]}")
        else:
            st.warning("Clear your sidebar filters to view the interactive
anatomy charts.")
```